Supervised Classification Algorithms

Week # 9

Course Instructor: Mahrukh Khan

Cross Validation in Machine Learning

- Cross-validation is crucial because it ensures that our model is not just memorizing the training data but generalizing well to unseen data. This process helps in:
- Reducing overfitting and underfitting.
- Providing a more accurate estimate of model performance.
- Ensuring the model is robust and reliable.

Types of Cross Validation Techniques

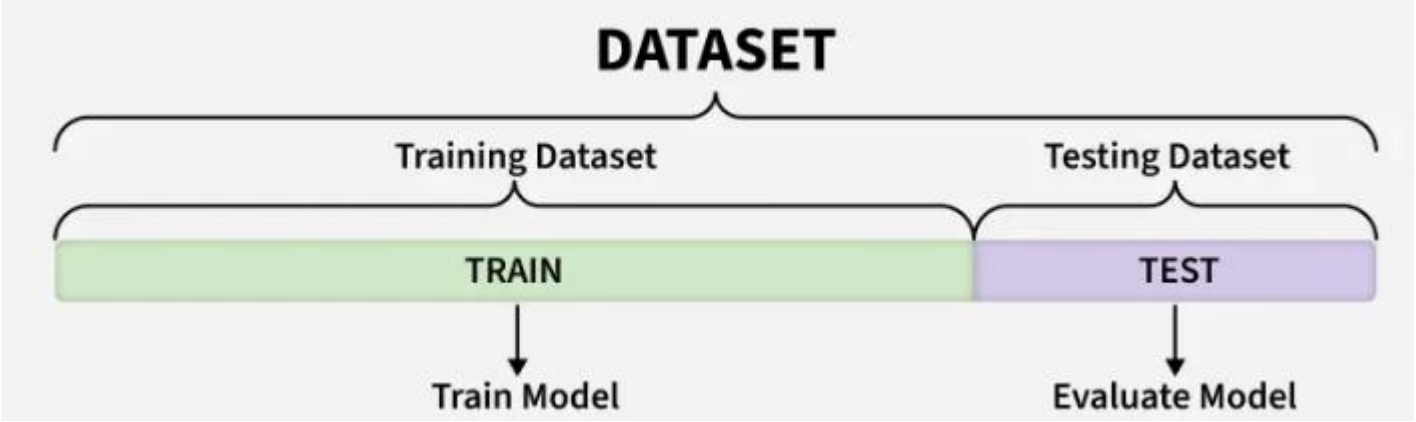
1. Holdout Validation

   In Holdout Validation method typically, some data is used for training and some for testing. Making it simple and quick to apply.

2. LOOCV (Leave One Out Cross Validation)
3. Stratified Cross-Validation
4. K-Fold Cross Validation

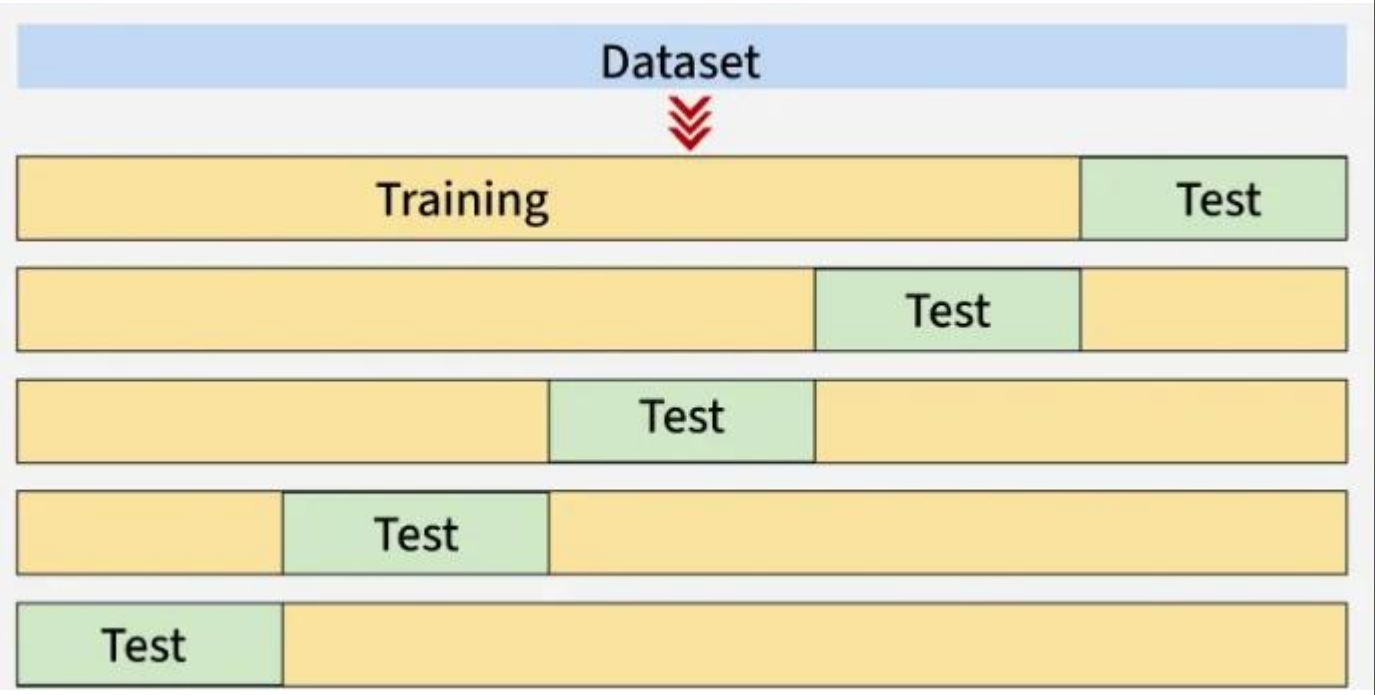
Hold Out

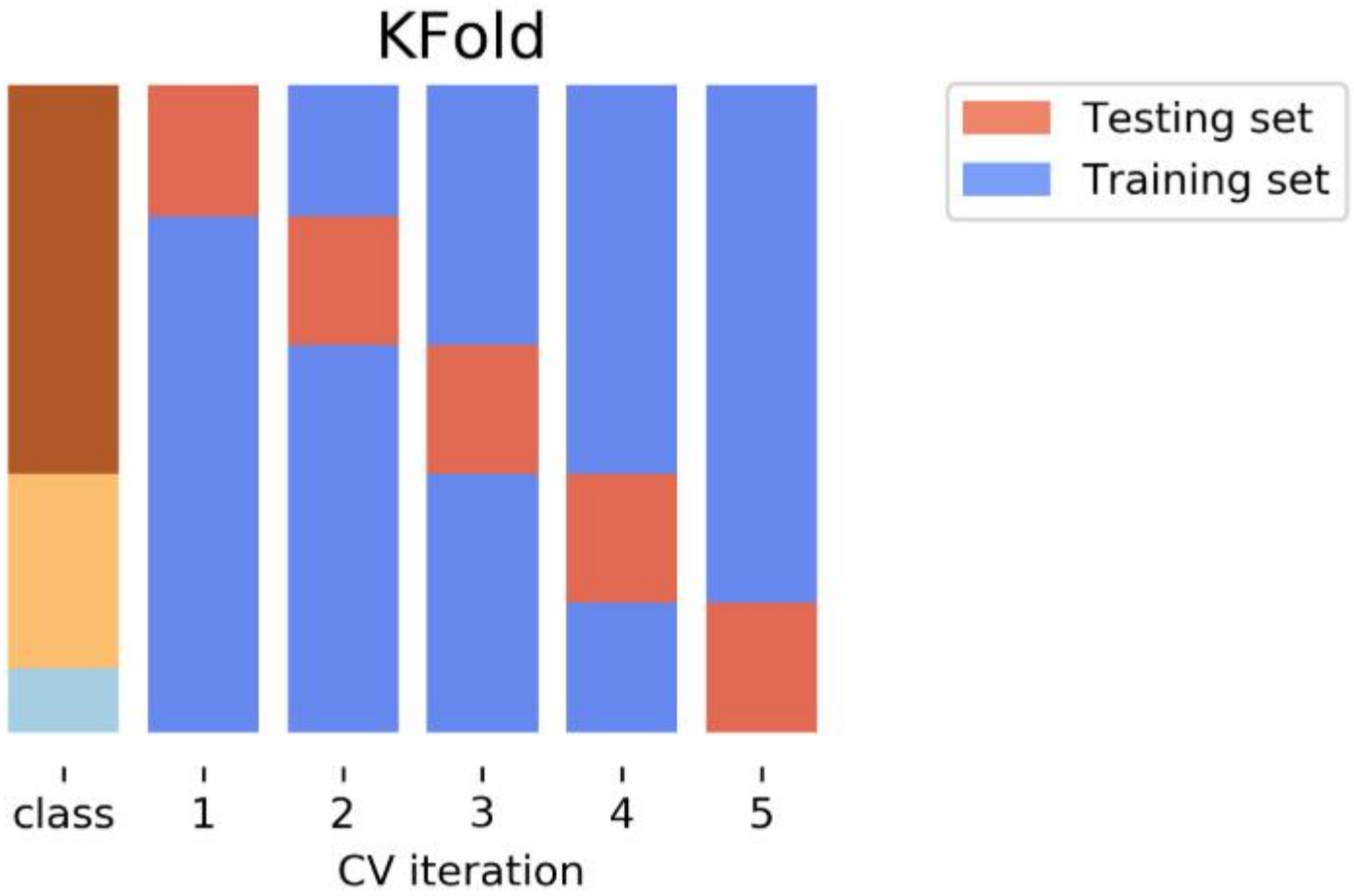
- The Holdout Method is a fundamental validation technique in machine learning used to evaluate the performance of a predictive model. In this method, the available dataset is split into two mutually exclusive subsets:
  - Typical split ratios include 70:30, 80:20 or 60:40 depending on dataset size.
- It is most effective when the dataset is large enough to allow meaningful splitting.

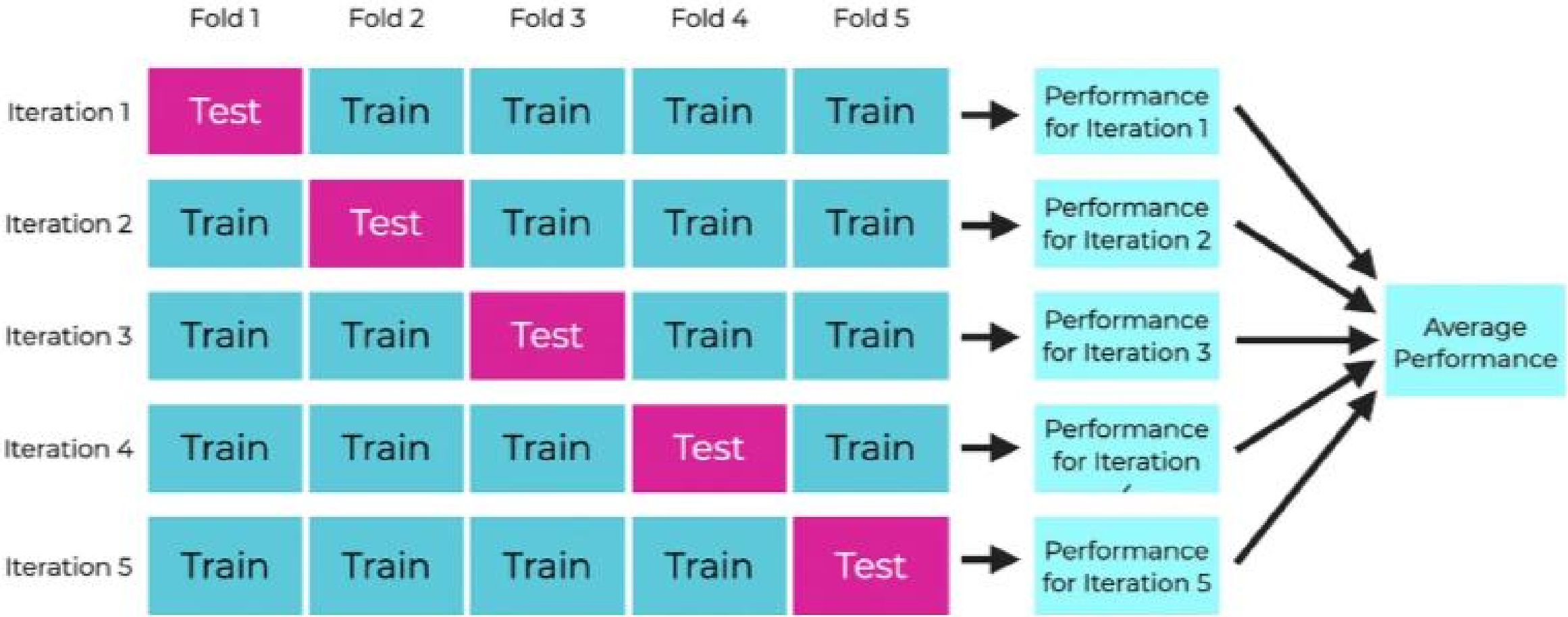


K-Fold Cross Validation

- K-Fold Cross Validation splits the dataset into *k* equal-sized folds. The model is trained on *k-1* folds and tested on the remaining fold. This process is repeated *k* times each time using a different fold for testing. For Final Score, average of all
- k test results.

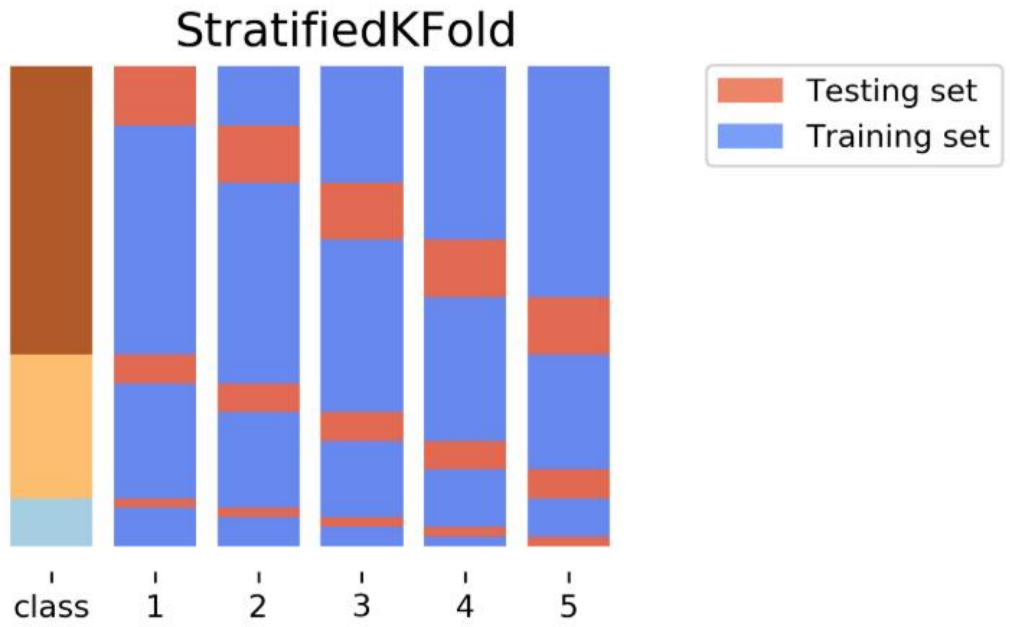



KFold
Testing set
Training set
class
1
2
3
4
5
CV iteration


Fold 1
Fold 2
Fold 3
Fold 4
Fold 5
Iteration 1
Test
Train
Train
Train
Train
Performance for Iteration 1
Iteration 2
Train
Test
Train
Train
Train
Performance for Iteration 2
Iteration 3
Train
Train
Test
Train
Train
Performance for Iteration 3
Iteration 4
Train
Train
Train
Test
Train
Performance for Iteration
Iteration 5
Train
Train
Train
Train
Test
Performance for Iteration 5
Average Performance

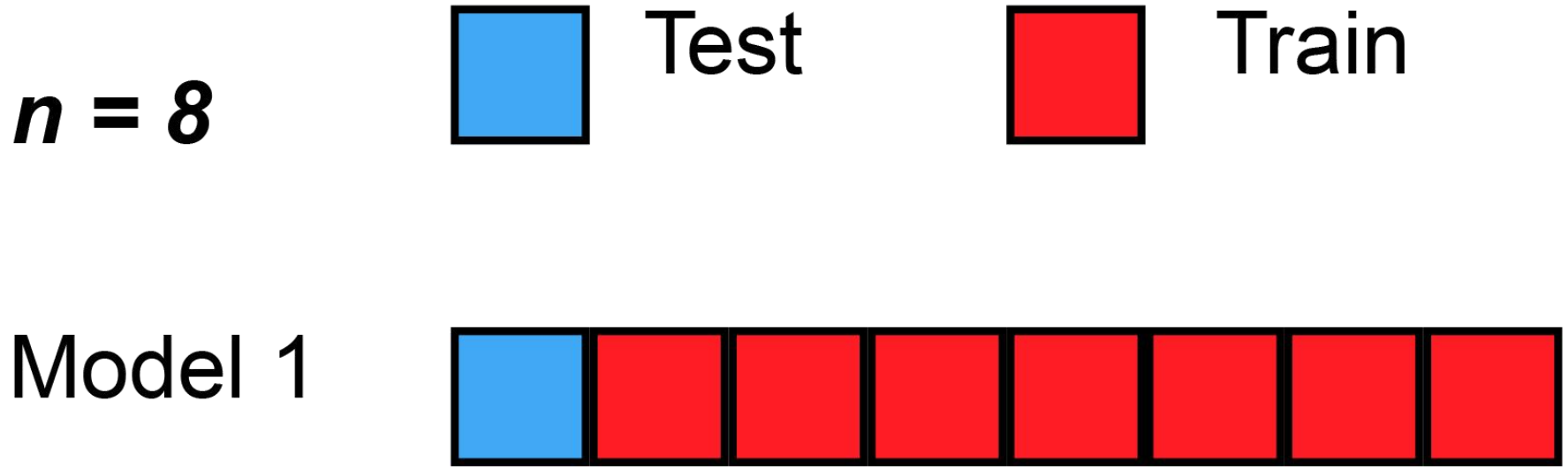
Stratified

- Similar to k-Fold but maintains **class proportion** (useful for imbalanced datasets). StratifiedKFold preserves the class frequencies in each fold to be the same as of the overall dataset.
- Stratification seeks to ensure that each fold is representative of all strata of the data.



LOOCV (Leave One Out Cross Validation)

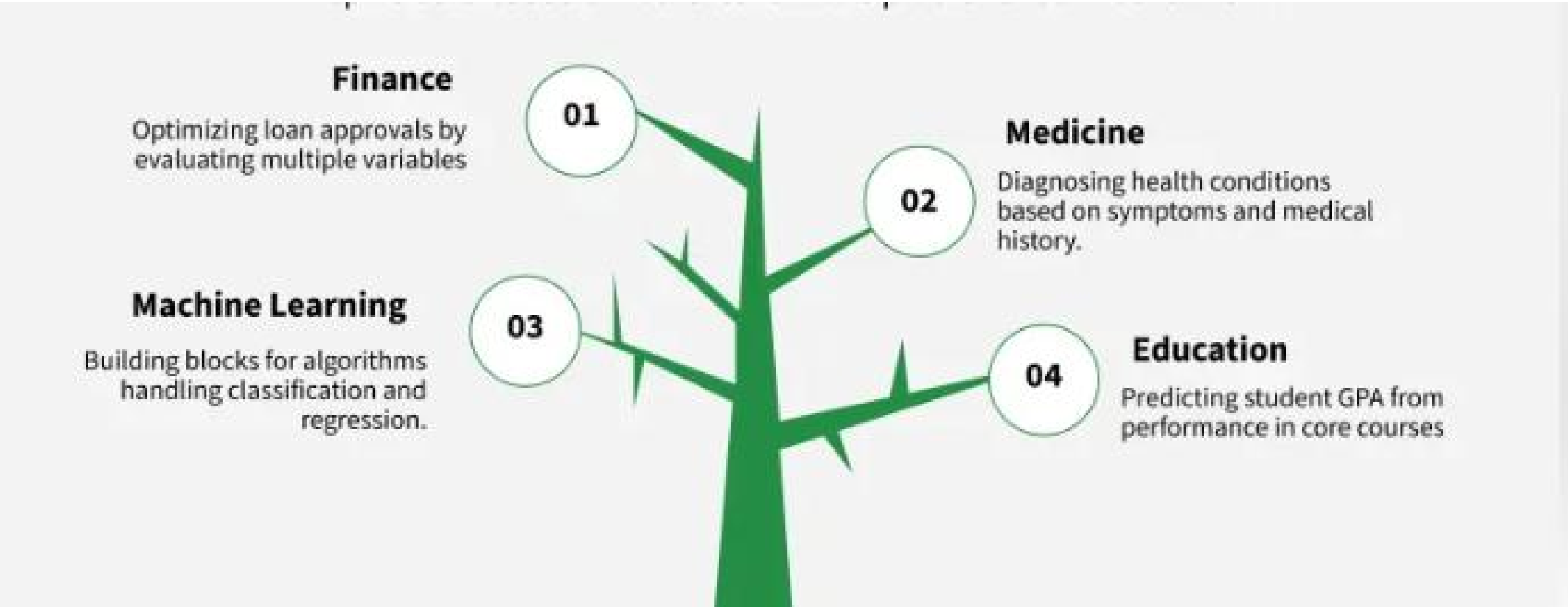
- In this method the model is trained on the entire dataset except for one data point which is used for testing. This process is repeated for each data point in the dataset.
- It can be very time-consuming for large datasets as it requires one iteration per data point.



Decision Trees

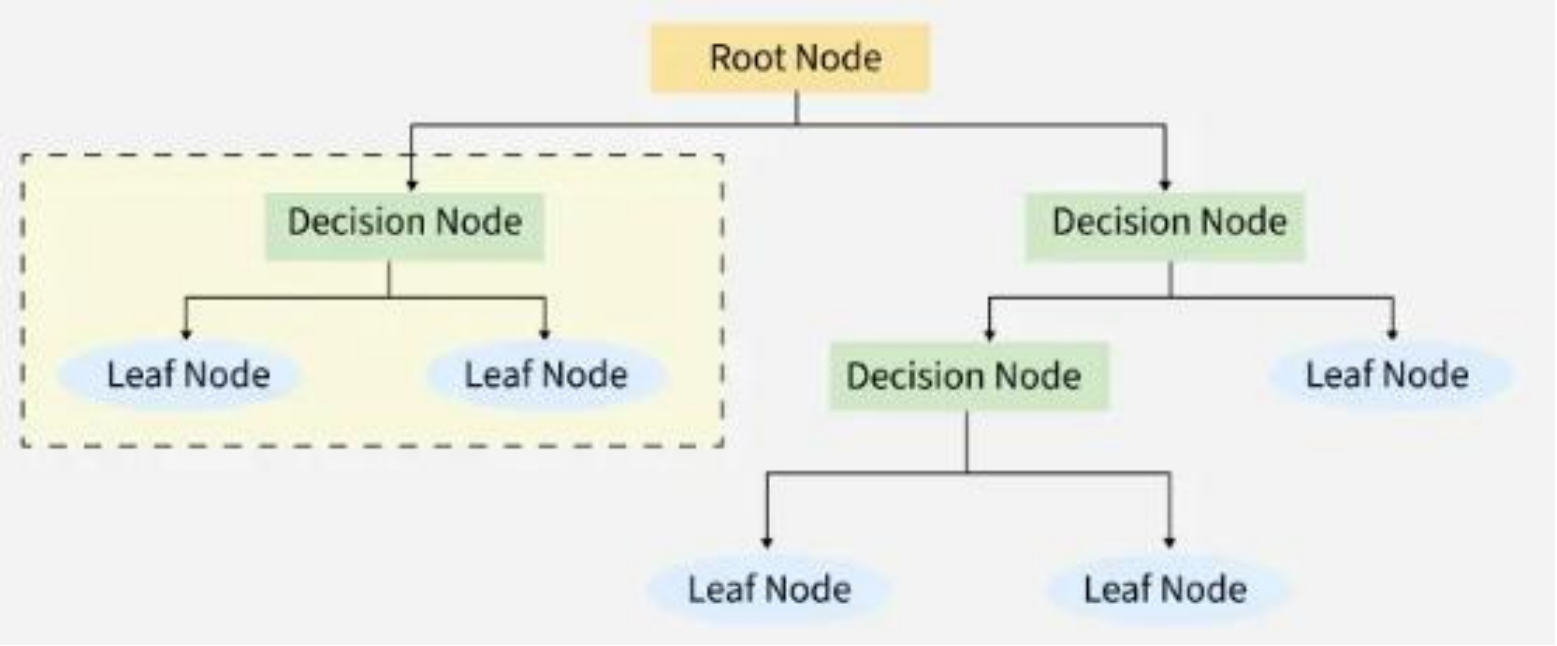
- A Decision Tree is a supervised machine learning algorithm used for both classification and regression tasks.
- It works by splitting data into branches based on feature conditions, forming a structure that looks like a tree 🌳
- A decision tree makes decisions by asking a series of questions about the input features.
- For example, to classify animals:
- Is it a mammal? → Yes → Does it lay eggs? → No → Do
- → Yes → Platypus
- → No → Bird

Applications of Decision Trees



Structure of Decision Tree

- Root Node → The starting point (the whole dataset).
- Decision Nodes → Points where data is split based on a feature.
- Leaf Nodes → Final outcomes (class labels or values).
- Branches → The conditions leading to decisions.

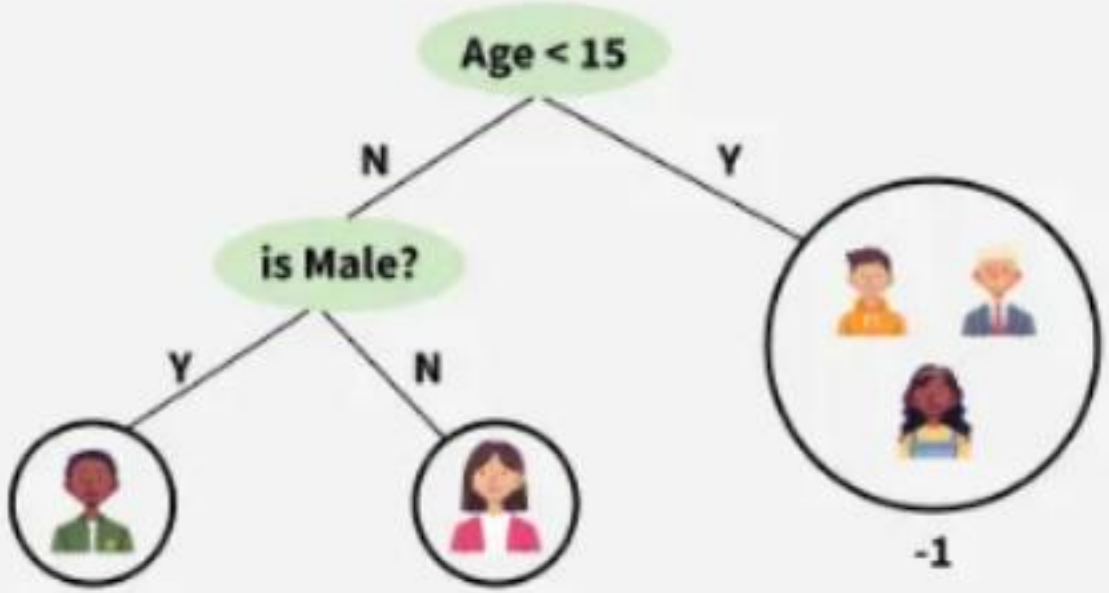


Working of Decision Tree

The model checks conditions like age and gender to split users into groups. Each group (leaf node) gets a prediction score based on user preferences for computer games.

Input: Age, Gender, Occupation,..

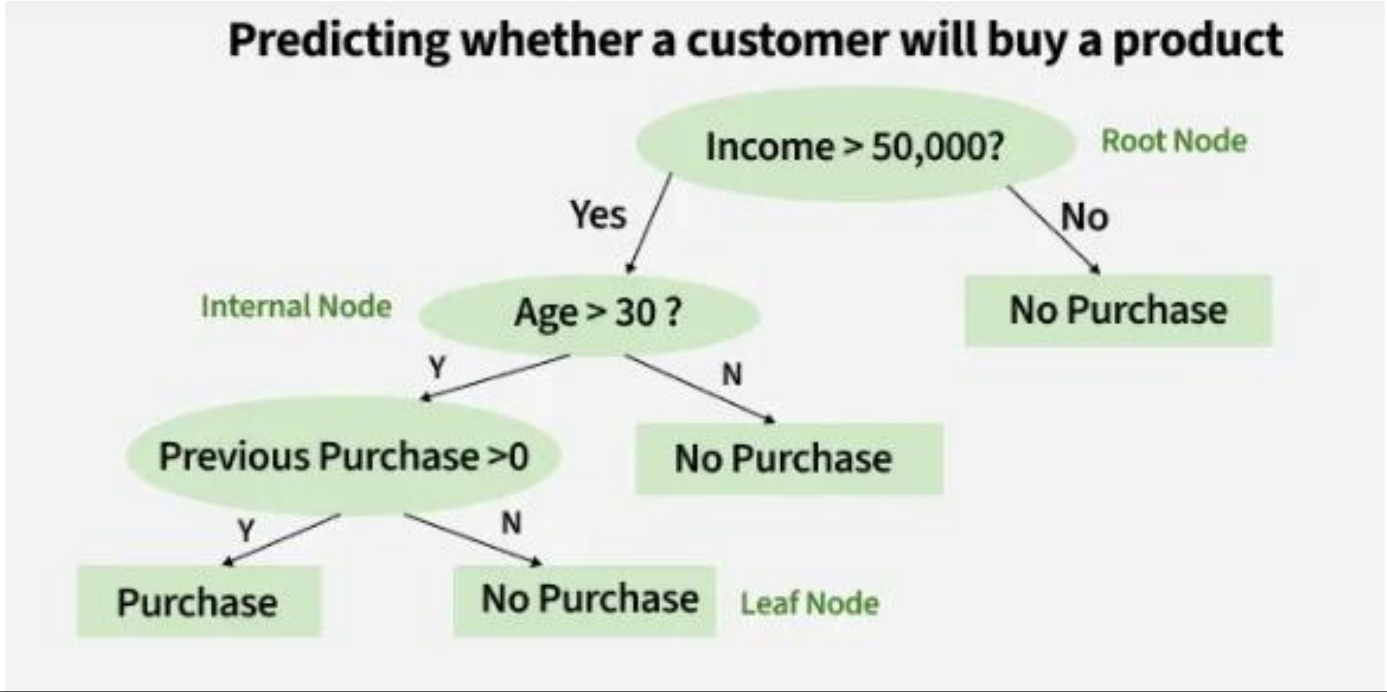
Does the person likes computer games

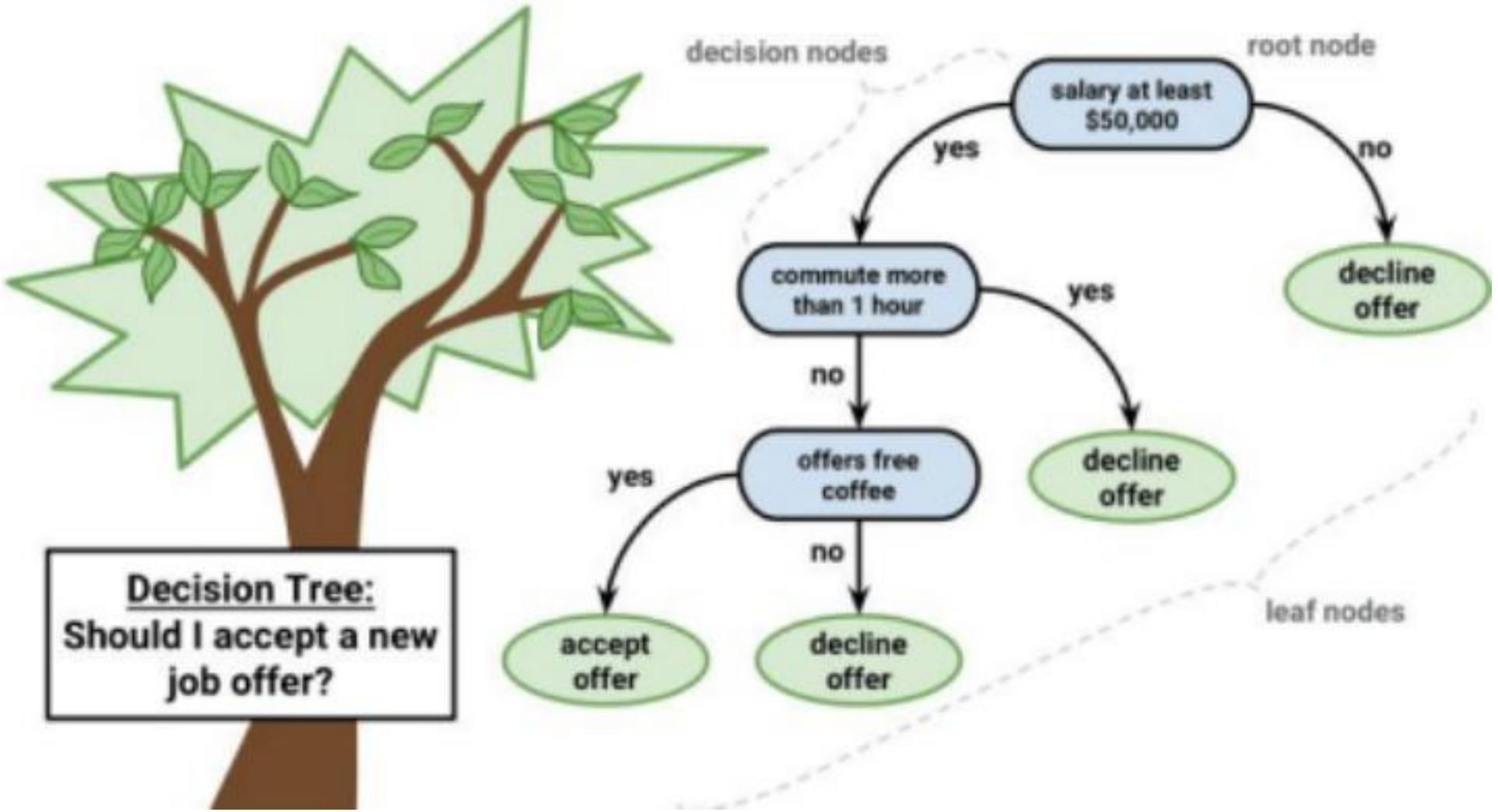


Prediction score in each leaf

How Does a Decision Tree Work?

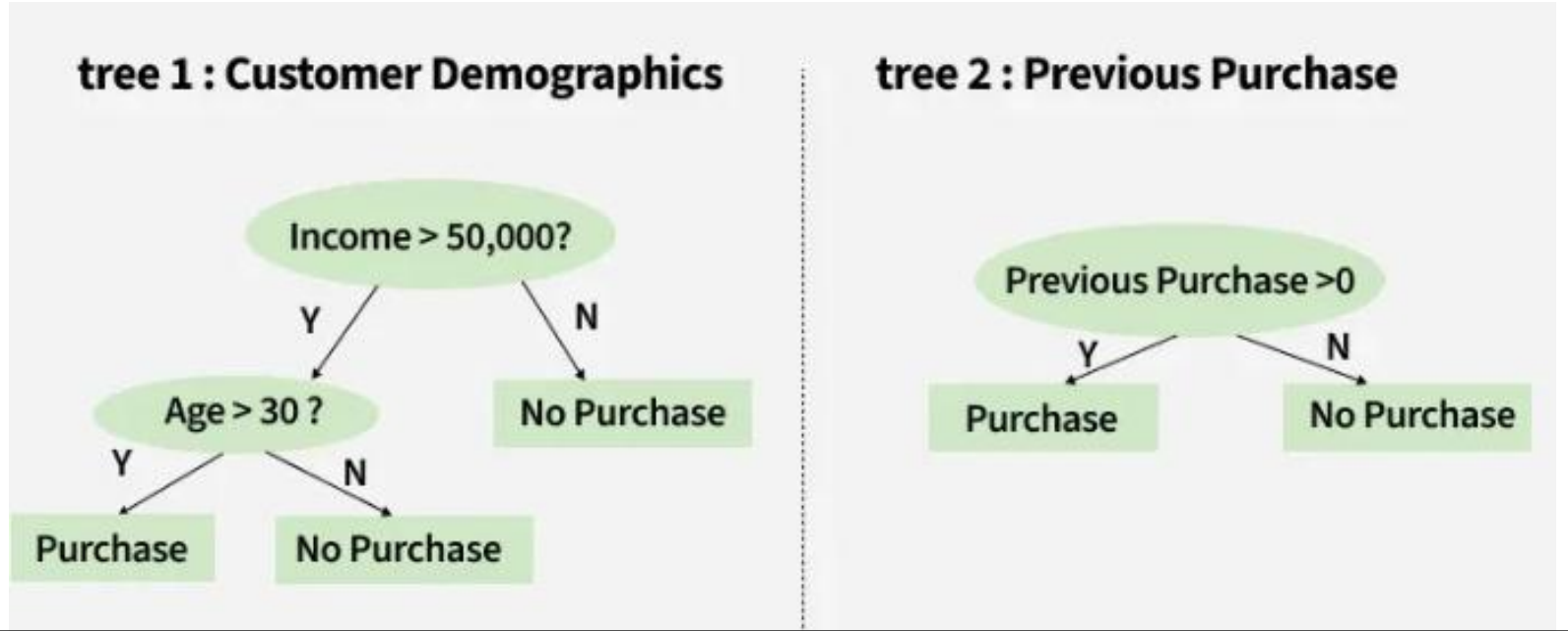
- A decision tree splits the dataset based on feature values to create pure subsets ideally all items in a group belong to the same class.
- Each leaf node of the tree corresponds to a class label and the internal nodes are feature-based decision points.




Decision Tree:
Should I accept a new job offer?
decision nodes
root node
salary at least $50,000
yes
no
commute more than 1 hour
yes
decline offer
no
offers free coffee
yes
decline offer
yes
no
accept offer
decline offer
leaf nodes

Example: Predicting Whether a Customer Will Buy a Product Using Two Decision Trees

- Once we have predictions from both trees, we can combine the results to make a final prediction. If Tree 1 predicts "Purchase" and Tree 2 predicts "No Purchase", the final prediction might be "Purchase" or "No Purchase" depending on the weight or confidence assigned to each tree. This can be decided based on the problem context.



Entropy

- “Entropy is a measure of **uncertainty** or **disorder** in data.
- **Example:**
  - If all students in class pass → no confusion → **entropy = 0**
    If half pass and half fail → high confusion → **entropy = maximum**
- High entropy → mixed data (unpredictable)
- Low entropy → pure data (predictable)

$$\text{Entropy}(S) = -p_1 \log_2(p_1) - p_2 \log_2(p_2)$$

- Where:
  - p_1 =proportion of positives
  - p_2 =proportion of negatives

Example:

Suppose in a dataset of 10 students: 6 passed, 4 failed $p_1 = 6/10 = 0.6, p_2 = 4/10 = 0.4$ $\text{Entropy} = -[0.6\log_2(0.6) + 0.4\log_2(0.4)] = 0.97$ ✓ High entropy → data is quite mixed (60–40 split)	If all 10 students pass: $p_1 = 1,\ p_2 = 0$ $\text{Entropy} = -[1\log_2(1) + 0\log_2(0)] = 0$ ✓ No disorder → entropy = 0 (perfectly pure)

Information Gain

- Information Gain tells us how useful a question (or feature) is for splitting data into groups.
- It measures how much the uncertainty decreases after the split.
  - For example, if we split a dataset of people into "Young" and "Old" based on age and all young people bought the product while all old people did not, the Information Gain would be high because the split perfectly separates the two groups with no uncertainty left.
- When we split the data (like in a Decision Tree), we check if the **uncertainty (entropy)** decreases.
- Information $Gain = Entropy(before\ split) - Weighted\ Entropy(after\ split)$

Gini Index

- like entropy, Gini measures **uncertainty**, but it's **simpler** and **computationally faster**.
- If all samples belong to **one class** → **Gini = 0** (pure node)
- If samples are **evenly split** between classes → **Gini = maximum**

$$\text{Gini}(S) = 1 - \sum_{i=1}^{n} p_i^2$$

Example-2

6-Yes **4-NO**	**All 10 are "Yes" →** $p(Yes) = 1$	**Evenly Mixed (50-50)**
p(Yes)=0.6,p(No)=0.4 $\text{Gini} = 1 - (0.6^2 + 0.4^2) = 0.48$	Gini=1−(12+02)=0	Gini=1−(0.52+0.52) =1−(0.25+0.25) =0.5
✓ **Gini = 0.48**	✓ **Gini = 0** → Pure node	✓ **Maximum impurity = 0.5** (for binary classification)

Splitting criteria in Decision Tree

- In decision tree, splitting criteria helps decide which features to split on at each node,

Gini Index

$$I_G = 1 - \sum_{j=1}^{c} p_j^2$$

P_j : proportion of the samples that belongs to class c for a particular node

Entropy

$$I_H = -\sum_{j=1}^{c} p_j log_2(p_j)$$

p_j: proportion of the samples that belongs to class c for a particular node.

*This is the the definition of entropy for all non-empty classes (p ne0) The entropy is 0 if all samples at a node belong to the same class.

Example

Label	Count	Probability
a	3	p(a) =3/8 = 0.375)
b	5	p(b) =5/8 = 0.625)

- *For the set X = {a,a,a,b,b,b,b,b}*
  Total instances: 8
  Instances of b: 5
  Instances of a: 3
- *Entropy(X)*=−[p(a)log2(p(a))+p(b)log2(p(b))]
  =−[0.375log2(0.375)+0.625log2(0.625)]
  =0.9544
- Gini=1−[p(a)2+p(b)2]
  - =1−[(0.375)2+(0.625)2]
  - =0.4688

Example

- Right now, we only have one set (X) — no attribute to split on — so **we can't compute IG yet**.
- we split X into two groups
- Step 1 ☐ — Entropy of each subset
- For S1, p(a)=2/3=0.667,p(b)=1/3=0.333

Subset	Elements	Count	a's	b's
S_1	{a, a, b}	3	2	1
S_2	{a, b, b, b, b}	5	1	4

- Entropy(S1)=−[0.667log2(0.667)+0.333log2(0.333)]=0.918
- For S2,
- p(a)=1/5=0.2,p(b)=4/5=0.8
- Entropy(S2)=−[0.2log2(0.2)+0.8log2(0.8)]=0.722

- Step 2 ☐ — Weighted Average Entropy (After Split)

$$Entropy_{after} = \frac{3}{8}(0.918) + \frac{5}{8}(0.722)$$

$$= 0.344 + 0.451 = 0.795$$

- Step 3 ☐ — **Information Gain**

$$IG = Entropy_{before} - Entropy_{after}$$

$$= 0.954 - 0.795 = 0.159$$

Example

- Given the following training data : 3 features and 2 classes,
- Which feature best split using Information gain.
- We will take each of the features and calculate the information for each feature.

X	Y	Z	C
1	1	1	I
1	1	0	I
0	0	1	II
1	0	0	II

- **Entropy(X=1)**
  = $-(2/3)\log_2(2/3) - (1/3)\log_2(1/3)$
  = 0.918
- **Entropy(X=0)**
- = $-(0) - (1)\log_2(1) = 0$

$$H(C|X) = \frac{3}{4}(0.918) + \frac{1}{4}(0) = 0.688$$

$$IG(X) = 1 - 0.688 = 0.312$$

- Best attribute to split = Y???

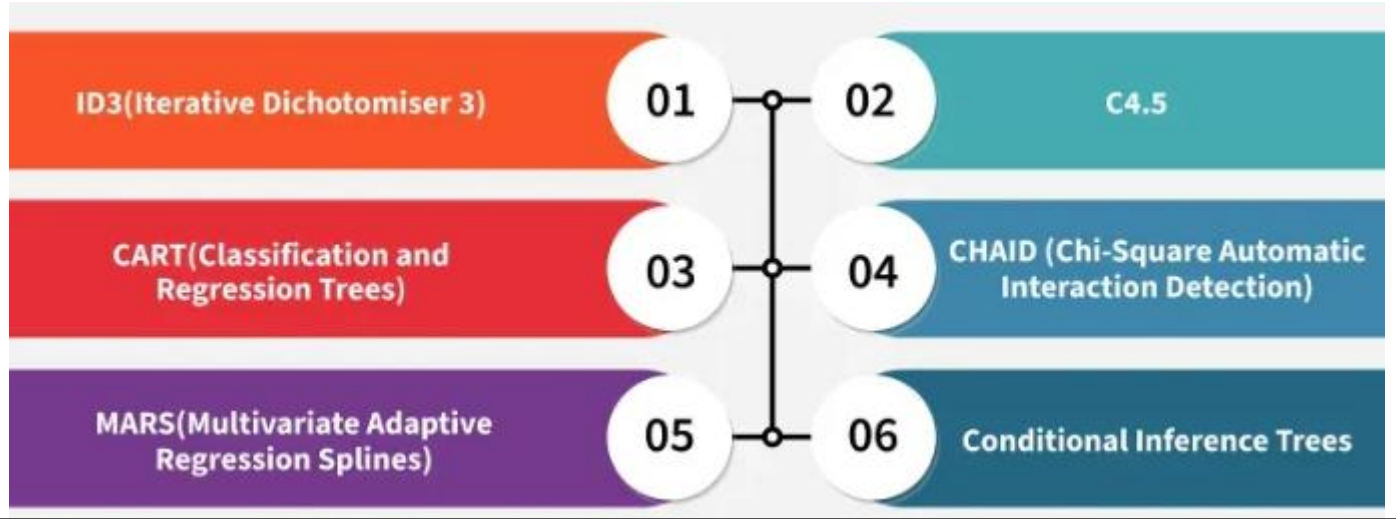
X	Count	Class I	Class II
1	3	2	1
0	1	0	1

Y	Count	Clas**s I	Class II
1	2	2	0
0	2	0	2

Z	Count	Class I	Class II
1	2	1	1
0	2	1	1

Types of Decision Tree

- **Classification Tree** and **Regression Tree**.
  There are 4 different types of splitting criteria :
- Categorical Target Variable (classification) : **Information Gain** , **Gini Impurity** and **Chi-Square**
- Continuous Target Variable (regression) : **Reduction in Variance**.



CART

- Classification and Regression Trees (CART) are a type of decision tree algorithm used for both classification and regression.
- splits a dataset into subsets based on the most significant variable. The goal is to create the purest subsets possible, where "pure" means that the subset contains only instances of a single class (for classification) or has minimal variance (for regression).

ID3-(Iterative Dichotomiser 3)

- ID3 is a classic decision tree algorithm commonly used for classification tasks. It works by greedily choosing the feature that maximizes the information gain at each node. It calculates entropy and information gain for each feature and selects the feature with the highest information gain for splitting.
- ID3 recursively splits the dataset using the feature with the highest information gain until all examples in a node belong to the same class or no features remain to split. After the tree is constructed it prune branches that don't significantly improve accuracy to reduce overfitting.

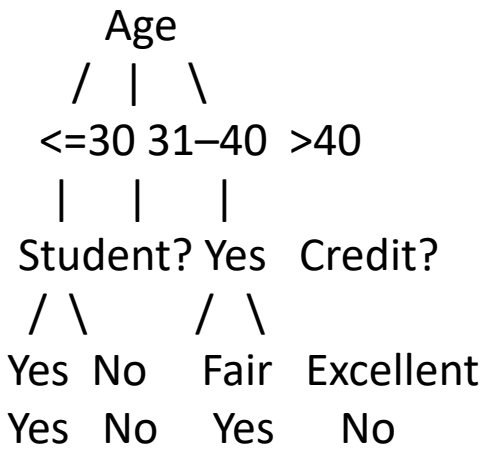
Decision Trees

Advantages	Disadvantages
Easy to understand and visualize	Sensitive to small changes in data
Handles both numerical and categorical data	Can be biased toward features with more levels
Requires little data preprocessing	Can overfit easily

Example-1

- Suppose we want to predict whether a person will buy a computer:

Age	Income	Student	Credit Rating	Buys Computer
<=30	High	No	Fair	No
<=30	High	No	Excellent	No
31–40	High	No	Fair	Yes
>40	Medium	Yes	Fair	Yes
>40	Low	Yes	Fair	Yes
<=30	Medium	Yes	Fair	Yes



Example

- **Step 1. Compute overall entropy**
  - Target variable: *Buys Computer*

$$Entropy(S) = -\frac{4}{6}\log_2\frac{4}{6} - \frac{2}{6}\log_2\frac{2}{6} = 0.918$$

- Step 2. Compute entropy for each attribute

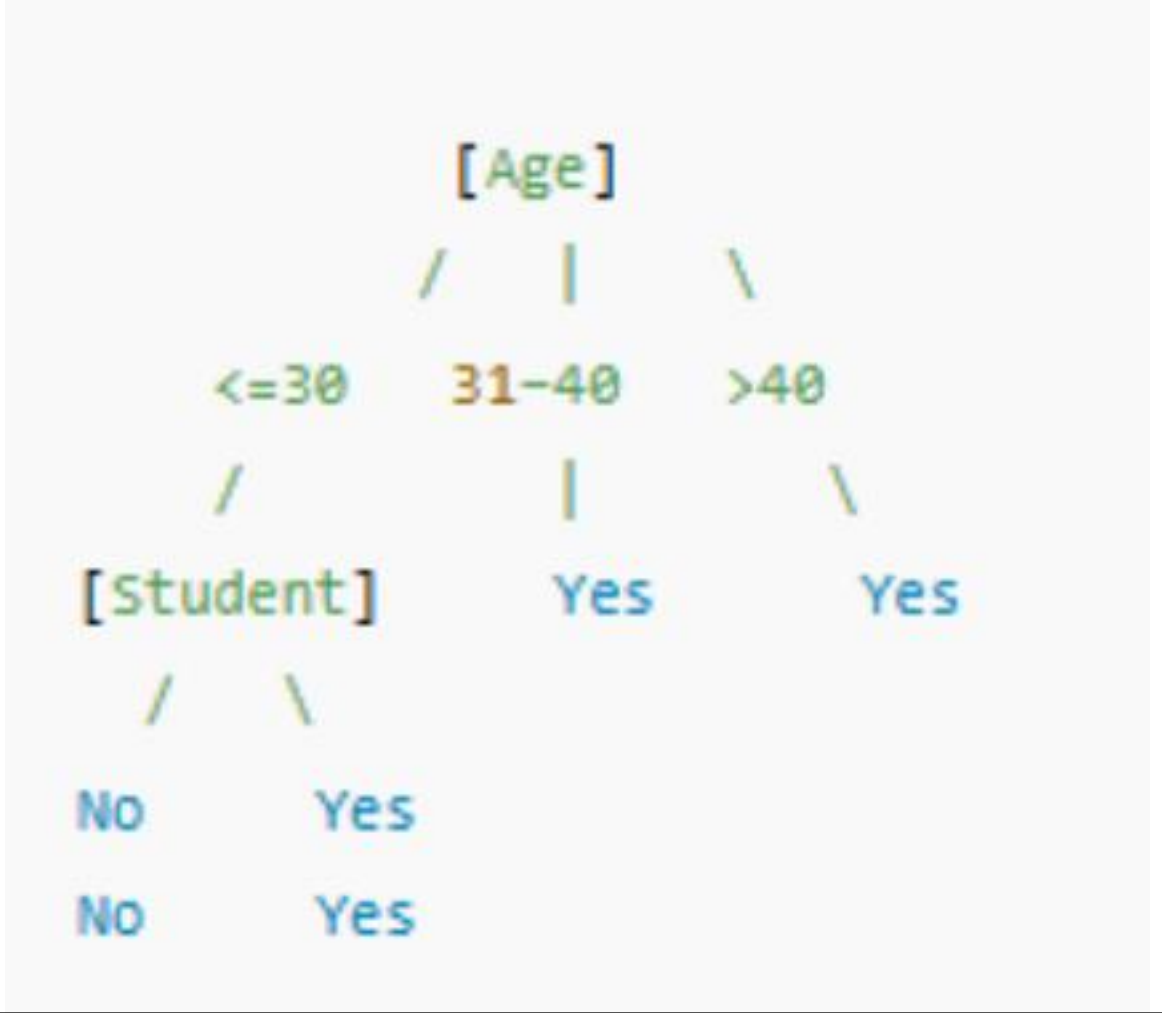
Age	Yes	No	Entropy
<=30	1	2	0.918
31–40	1	0	0
>40	2	0	0
Income	Yes	No	Entropy
High	1	2	0.918
Medium	2	0	0
Low	1	0	0

Student	Yes	No	Entropy
Yes	3	0	0
No	1	2	0.918

Credit	Yes	No	Entropy
Fair	4	1	0.722
Excellent	0	1	0

For Age, Weighted entropy: $\frac{3}{6}(0.918) + \frac{1}{6}(0) + \frac{2}{6}(0) = 0.459$ **Information Gain(Age) = 0.918 – 0.459 = 0.459**	**For Income, Weighted entropy:** $\frac{3}{6}(0.918) + \frac{2}{6}(0) + \frac{1}{6}(0) = 0.459$ **Information Gain(Income) = 0.918 – 0.459 = 0.459**
For Student, Weighted entropy: $\frac{3}{6}(0) + \frac{3}{6}(0.918) = 0.459$ **Gain(Student) = 0.918 – 0.459 = 0.459**	For Credit Rating, Weighted entropy: $\frac{5}{6}(0.722) + \frac{1}{6}(0) = 0.602$ **Gain(Credit Rating) = 0.918 – 0.602 = 0.316**

- If **Age ≤ 30** and **Student = Yes** → **Buys = Yes**
- If **Age ≤ 30** and **Student = No** → **Buys = No**
- If **Age = 31–40** → **Buys = Yes**
- If **Age > 40** → **Buys = Yes**



Same Example using GINI

- **Step 1: Gini for the whole dataset**
  - Yes = 4,No = 2
  - $Gini(S) = 1 - \left(4/6(^2 - \left(2/6(^2 = 1 - 0.444 - 0.111 = 0.445$
- **Step 2: Gini for each attribute**

Age	Yes	No	Gini
<=30	1	2	1 - (1/3)² - (2/3)² = 0.444
31–40	1	0	0
>40	2	0	0

Income	Yes	No	Gini
High	1	2	0.444
Medium	2	0	0
Low	1	0	0

Student	Yes	No	Gini
Yes	3	0	0
No	1	2	0.444

Credit	Yes	No	Gini
Fair	4	1	0.32
Excellent	0	1	0

$(Gini(Age) = (3/6(0.445) - 1/6(0) + 2/6(0) = 0.222$

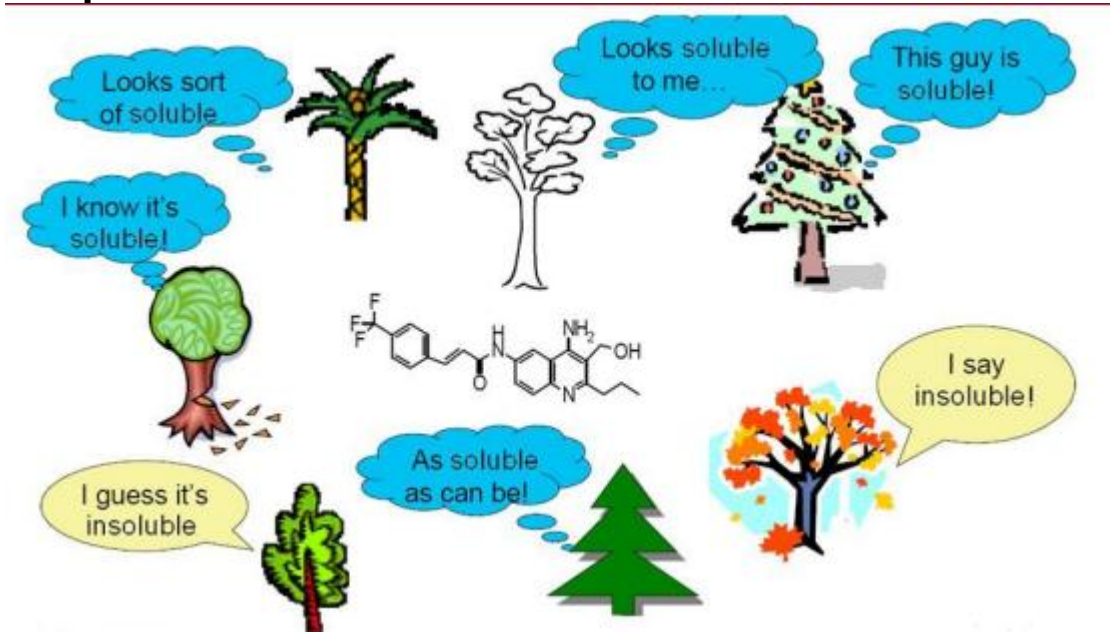
Reduction = 0.445 - 0.222 = 0.223

- **Step 3: Compare reductions**
- **Step 4: CART tie-breaking**
  - All three (Age, Income, Student) give the **same Gini reduction**.
    CART picks the **first** in dataset order → **Age**.

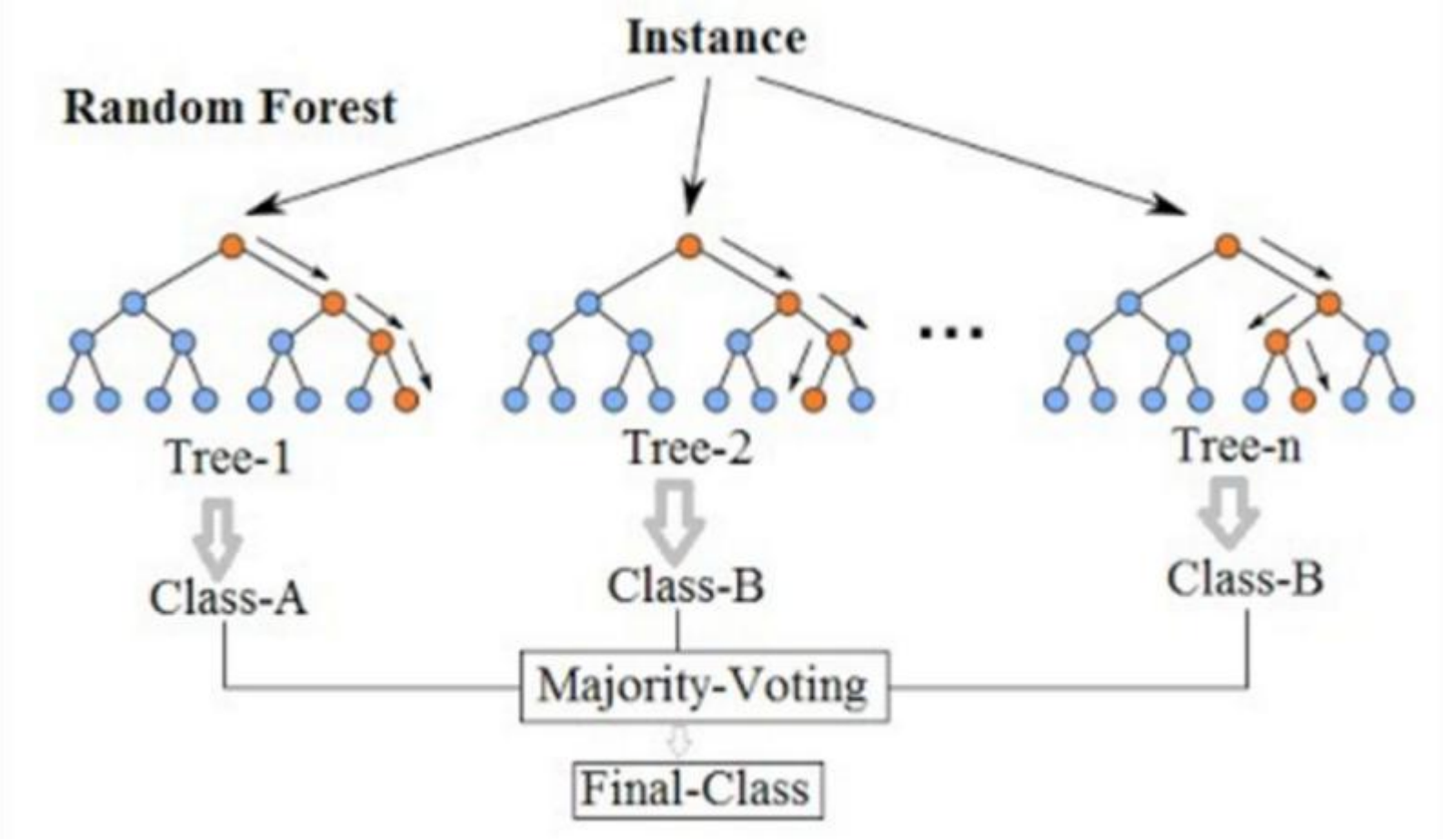
Attribute	Gini Reduction
Age	0.223
Income	0.223
Student	0.223
Credit Rating	0.179

Random Forest

- A Random Forest is an ensemble (collection) of Decision Trees working together.
- Instead of relying on a single decision tree, it builds many trees and combines their outputs to make a final prediction.
- Introduced by Leo Breiman in 2001
- It’s like:One tree = one opinion
- Random Forest =
  a group vote → majority wins



Random Forest

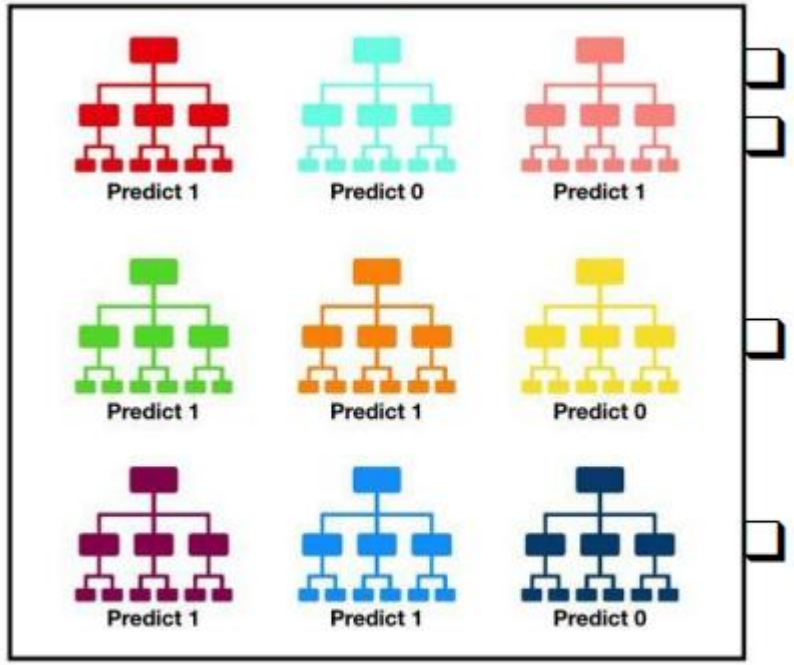


Assumptions of Random Forest

- **Handles Missing Data:** It can work even if some data is missing so you don’t always need to fill in the gaps yourself.
- **Shows Feature Importance:** It tells you which features (columns) are most useful for making predictions which helps you understand your data better.
- **Works Well with Big and Complex Data:** It can handle large datasets with many features without slowing down or losing accuracy.

Working of Random Forest Algorithm

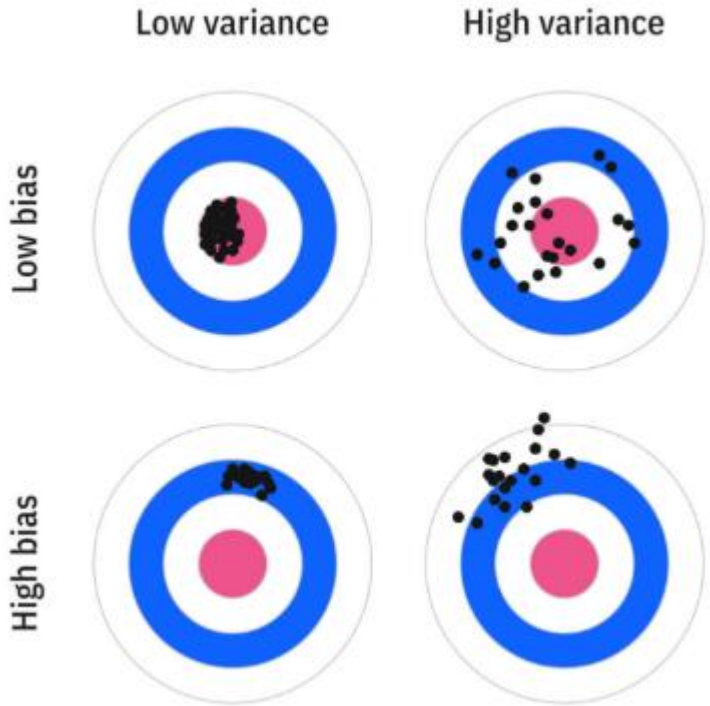
- **Create Many Decision Trees:** The algorithm makes many decision trees each using a random part of the data. So every tree is a bit different.
- **Random Feature Selection**
  - At each split, only a random subset of features is considered.(This adds diversity among trees.)
- **Train Many Trees**
  - Each tree makes its own prediction.
- **Combine Predictions**
  - Classification → majority vote
  - Regresion → average of predictions



Bias-Variance

- **Bias** measures how far off, on average, a model's predictions are from the ground truth values. High-bias models tend to make strong assumptions about the form of the data and cause underfitting. An overly simplistic model tends to have high bias and low variance—a model like this tends to have high training errors and high prediction errors.
- **Variance** measures how much a model's predictions change with different training datasets. High-variance models are sensitive to noise in the training data and cause overfitting. A model with complex architecture and more parameters tends to have high variance and low bias.

Scenario	Bias	Variance	Explanation
All arrows far from target but close to each other	High	Low	Consistently wrong → **underfitting**
Arrows scattered around target	Low	High	Sometimes right, sometimes wrong → **overfitting**
Arrows tightly around bullseye	Low	Low	**Ideal model**



Striking the right balance is at the heart of effective machine learning design and helps explain why models that perform well on training data might still fail in the real world.

Underfitting vs Overfitting

- Overfitting refers to a scenario when the model tries to cover all the data points present in the given dataset. It starts caching noise and reduces the efficiency and accuracy of the model. The overfitted model has low bias and high variance.
- Underfitting occurs when our model is not able to capture the underlying trend of the data. An underfitted model has high bias and low variance.
- Example:
  - Underfitting = “studied too little, can’t answer exam questions.”
  - Overfitting = “memorized past papers, fails on new exam.”

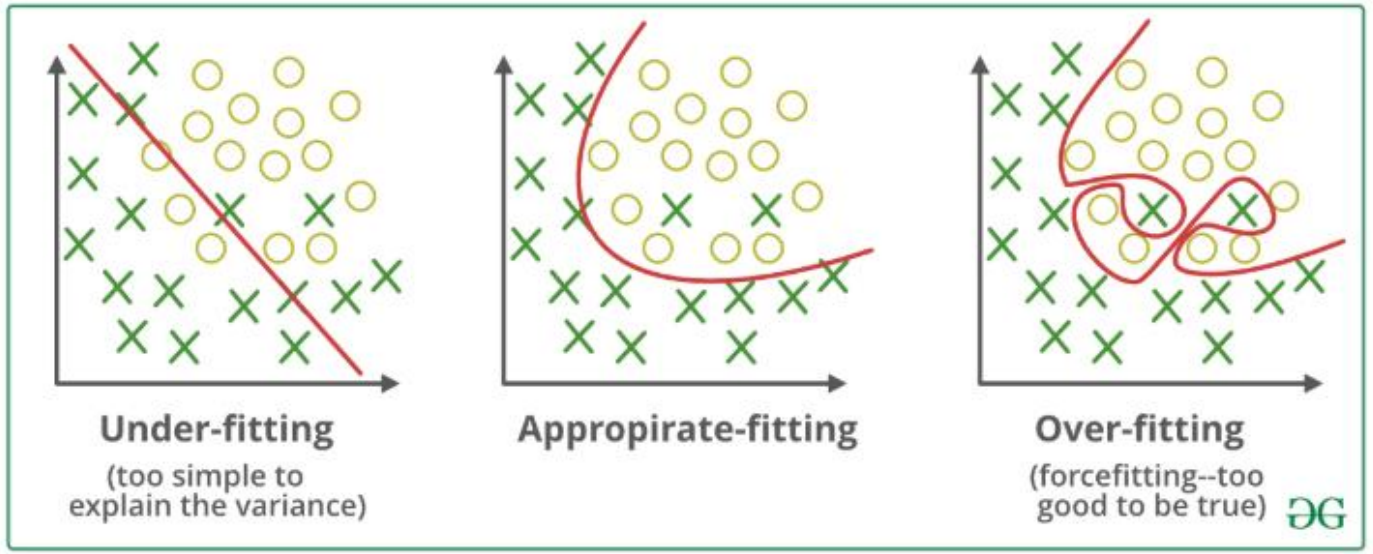
How to avoid Overfitting

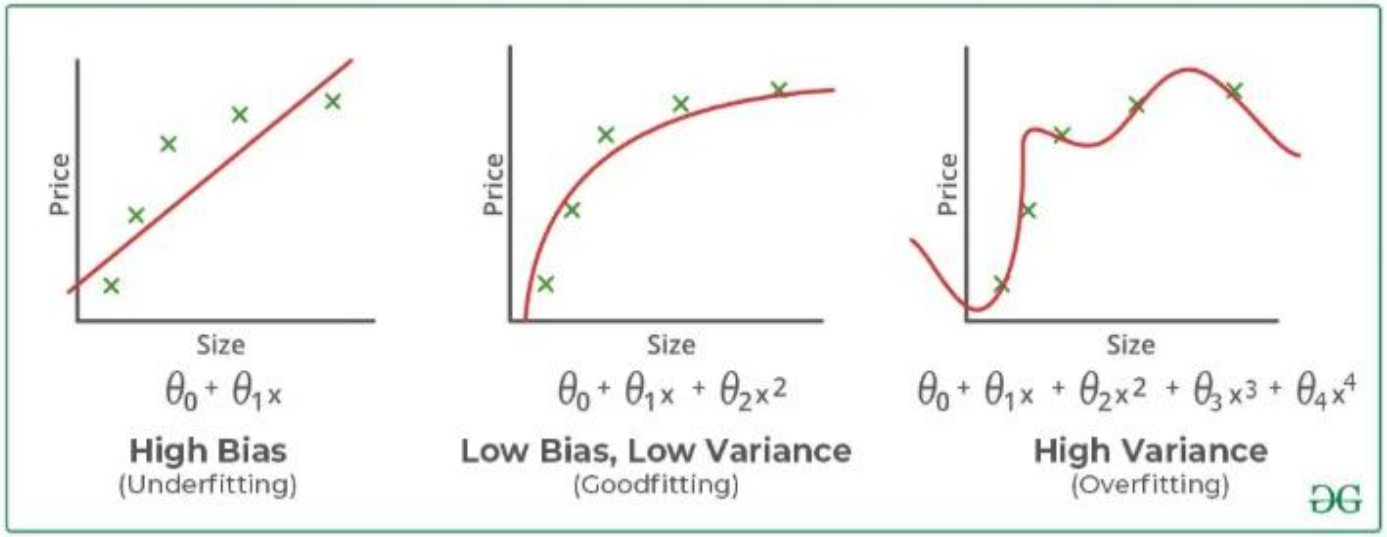
- Using cross-validation
- Using Regularization techniques
- Implementing Ensembling techniques.
- Picking a less parameterized/complex model
- Training the model with sufficient data
- Removing features
- Early stopping the training

How to avoid Underfitting

- Preprocessing the data to reduce noise in data
- More training to the model
- Increasing the number of features in the dataset
- Increasing the model complexity
- Increasing the training time of the model to get better results.

Underfitting vs Overfitting





In theory, the **ideal model** has:

- **Low bias** → it correctly captures the true underlying pattern
- **Low variance** → it's stable and doesn't fluctuate wildly with small data changes

- "The goal isn't to eliminate bias or variance — it's to *balance* them where total error is lowest."

$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$